# column-hse-ltau: implementation

Technical notes on `run.mode = "column-hse-ltau"`. HSEMHS, 24 Oct 2026.

## 1. Running it

```
[mpiexec -n R] julia --project=. -t T scripts/run.jl config/modes/column-hse-ltau.toml
```

Driver `run_column_hse_ltau` (`src/RunModes.jl`); physics in `src/PoissonTau.jl`; MPI layer in `src/Slab.jl`.

| Key | Default | Meaning |
|---|---|---|
| `input.file` | required | $\tau$-grid cube (layout below) |
| `input.x`, `input.y` | `x`, `y` | names of the coordinate datasets |
| `physics.g` | `"from_input"` | number, or the file attribute `gravity_m_s2` (error if absent) |
| `eos.table_file` | `cache/eos_table_fullz.h5` | EOS and opacity table |
| `tau_grid.cap_per_dex` | 50 | largest rise of $\ln P$ per dex in $\tau$ |
| `tau_grid.align` | `ptot` | `ptot`: equal $P + B^2/2\mu_0$ at one height; `none`: $z(\tau{=}1) = 0$ per column |
| `run.output`, `run.overwrite` | required, false | output file |

## 2. Input layout

`IO/input-mhs-ltau.nc`: `log10tau500` ($n_\tau$, increasing: top level first), `x` ($N_x$), `y` ($N_y$), `temperature`, `B_x`, `B_y`, `B_z` with h5py shape $(x, y, \tau)$, i.e. Julia $(\tau, y, x)$; SI (a `units` attribute on `x`, `y` is honoured). The reader also accepts the `make_tau_input.py` test layout (`logtau500`, `x_m`, `y_m`, `T`, `Bx`…, h5py $(\tau, y, x)$). Each rank reads only its $x$ hyperslab `f[name][:, :, i0:i1]`. Missing `x`/`y`, length mismatches, a non-increasing $\log\tau$ or a non-positive grid spacing are rejected as `invalid_input`.

## 3. Algorithm

1. `hse_tau!` per column, top down. Top: fixed point $\ln P \leftarrow \frac{1}{2}\ln P + \frac{1}{2}\ln(g\tau_1\rho/\chi)$ from $P = 100\,\mathrm{Pa}$ to $10^{-10}$. Between levels $a = \ln\tau_k$, $b = \ln\tau_{k+1}$: Heun (explicit trapezoid) substeps of $(\ln P, z)$ with $f = g\tau\rho/(\chi P)$, $g_z = -\tau/\chi$ and $\ln T$ linear in $\ln\tau$; step $h = \min(b - x, 0.05/|f|)$. Cap: if $\ln P$ would exceed $\ln P_k$+`cap_per_dex`$(\log\tau_{k+1} - \log\tau_k)$, or the table edge (`tab.Y[end]` $-1$), or $f$ is not finite, the level gets the capped value and the interval is counted. $z$ is shifted so that $z(\log\tau = 0) = 0$ (linear in $\log\tau$, `interp_lt`).
2. `align_ptot!`: reference $\ln(P + B^2/2\mu_0)$ = global median over the bottom level (exact distributed median); per column the height where the total pressure crosses it (linear between levels, or hydrostatic extrapolation $z_{\mathrm{bot}} - \Delta\ln P_t/(g\rho/P)$ below the bottom), subtracted.
3. `gauge!`: subtract the global mean of $z(\log\tau = 0)$.
4. $\rho$ from the EOS at every node; output.

Any non-finite $\ln P$ after the integration stops the run ("left the EOS table").

## 4. MPI layer (`Slab.jl`)

Shared with the two Poisson modes.

- `slab_par`: `init_parallel`$(N_x, N_y, n;\ p_x = R,\ p_y = 1)$: each rank owns a contiguous range of $x$ columns and all of $y$ (at least two columns per rank). Arrays are $(k, j, i)$ = (vertical, $y$, $x$), columns contiguous.
- Reductions: `gsum`, `gdot`, `gnorm`, `gmax`, `gcount` (Allreduce).
- `gquantiles`/`gmedians`: *exact* global order statistics without gathering. Each Float64 is mapped to an order-preserving Int64 key (sign bit set: flip the magnitude bits); per set the $k$-th smallest key is found by bisection on the key range, one Allreduce of the per-set counts per round (at most 64 rounds), all sets at once. Type-7 quantiles as `Statistics.quantile`. Non-finite values are skipped.
- `coop_write`: rank 0 creates every dataset at its global size (Julia $(x, \dots)$, h5py $(\dots, x)$) and the attributes, then the ranks write their $x$ hyperslabs in turn between barriers (no parallel HDF5).
- The column integration has no communication: column results are bitwise identical for any rank count; the alignment and gauge use global reductions (median exact; mean to rounding).

Threads: owned columns in a threaded loop over $x$.

## 5. Output

`/tau_grid/{logtau500, z_m, Pgas, rho, T, Bx, By, Bz, z_tau1_m}` and `/coords/{x_m, y_m}`; arrays Julia $(x, y, \tau)$, h5py $(\tau, y, x)$. Attributes: `mode`, `input`, `eos_table`, `g`, `align`, `cap_per_dex`, `capped_intervals`, `mpi_ranks`, `threads`, `units`.

## 6. Verification and performance

- $96^2$ crop: 1 and 4 ranks give identical pressures; heights agree to $2 \times 10^{-10}$ m (the gauge mean).
- Full cube against the original script (`scripts/poisson/poisson_tau.jl`, `MHS_TAU_ITERS=0`, one rank): identical pressures, heights within 0.13 mm.
- Full cube ($504 \times 504 \times 95$), 8 ranks $\times$ 32 threads on 4 nodes: 95 s wall (most of it start-up and I/O), 1.9 GB per rank; 2585 capped intervals.